

ANALYSTLAB AFRICA EXPERIENCE LAB

HealthConnect Clinic — Week 6

ML Pipeline Integration & Validation Report

Intern Name: Grai Rudolf
Track: Machine Learning Engineering
Role: Junior Machine Learning Engineer
Date: September 2026

Project: HealthConnect Clinic
Document Version: 1.0
Stage: Integration & Validation
Status: Pre-Testing Deliverable

Abstract

This document presents the Week 6 ML Engineering track deliverable: the integration and validation of the HealthConnect ML pipeline. Building on the Week 5 modular pipeline, we add input/output validation checks, structured logging, improved (tuned) model configurations, and an end-to-end integrated pipeline class. Cross-track collaboration with the Data Science track provided candidate model parameters and error analysis that directly informed model integration. All 17 tests (12 unit + 5 integration) pass. The pipeline is now ready for broader testing in Week 7.

Contents

1	Week 5 → Week 6 Transition	2
1.1	Week 5 Main Output	2
1.2	Most Important Result	2
1.3	Main Limitation Discovered	2
1.4	Relevant Tracks for Week 6	2
1.5	What Week 6 Improves	2
2	Week 6 Pipeline Architecture	2
2.1	Repository Structure	2
2.2	New Modules (Week 6)	3
3	Validation Checks	3
3.1	Input Validation	3
3.2	Output Validation	3
4	Integrated Pipeline (HealthConnectPipeline)	4
4.1	Usage	4
4.2	Prediction Output Schema	4
5	Improved Model Configurations	4
6	Results & Model Comparison	5
6.1	Best Model (Held-Out Test Set)	5
7	Cross-Track Collaboration	5
8	Testing Evidence	6
9	Assumptions, Limitations, Risks & Dependencies	7
9.1	Resolved from Week 5	7
9.2	Remaining	8
9.3	New Issues Discovered	8
10	Recommended Focus for Week 7	8

1 Week 5 → Week 6 Transition

1.1 Week 5 Main Output

The Week 5 deliverable was a modular ML pipeline package (`pipeline/`) with separate modules for configuration, data processing, feature engineering, model training, and evaluation. A Jupyter notebook executed the full baseline workflow and five baseline classifiers were benchmarked.

1.2 Most Important Result

The primary baseline (Logistic Regression) achieved held-out recall 0.650, accuracy 0.617, and ROC-AUC 0.664, confirming that pre-appointment features carry meaningful predictive signal.

1.3 Main Limitation Discovered

The Week 5 pipeline lacked input/output validation checks, used raw `print()` statements instead of structured logging, had no end-to-end integration class (the pipeline was a sequence of separate function calls), and had no integration tests verifying that preprocessing and model components interact correctly.

1.4 Relevant Tracks for Week 6

The Data Science track produces the candidate model and error analysis that the ML Engineering pipeline must integrate. The Data Analytics track provides analytical findings that could inform feature refinement.

1.5 What Week 6 Improves

Week 6 adds validation, logging, an integrated pipeline class, improved (tuned) models, and integration testing — moving from individual components to a validated, integrated pipeline.

2 Week 6 Pipeline Architecture

2.1 Repository Structure

```

week 6/
  pipeline/
    __init__.py
    config.py           Paths, logging, validation schema
    data_processing.py  load, clean, preprocessing transformer
    feature_engineering.py
    model_training.py   Baseline + improved model definitions
    model_integration.py End-to-end integrated pipeline class
    evaluation.py       Metrics & visualisations
    validation.py       Input/output validation checks
    utils.py           Seeds & helpers
  tests/
    test_pipeline.py    Unit tests (12 tests)
    test_integration.py Integration tests (5 tests)
  figures/             Diagnostic figures
  reports/             This report

```

```

week6_healthconnect_ml_pipeline_integration.ipynb
README.md
requirements.txt

```

2.2 New Modules (Week 6)

- **validation.py:** Schema and output validation checks applied at each pipeline boundary.
- **model_integration.py:** `HealthConnectPipeline` class wrapping the complete load → clean → engineer → preprocess → train → predict flow into a single callable object.
- **config.py (updated):** Added structured logging configuration, validation schema (`REQUIRED_RAW_COLS`) and model registry constants.
- **model_training.py (updated):** Added `get_improved_models()` with tuned hyperparameters informed by Data Science error analysis.
- **data_processing.py (updated):** Replaced all `print()` calls with structured `logger.info()` calls.

—

3 Validation Checks

3.1 Input Validation

Table 1: Validation Checks Added in Week 6

Check	When	What It Validates
<code>validate_raw_data</code>	After load	All 18 required columns present; ≥ 100 rows
<code>validate_clean_data</code>	After cleaning	Binary target (0/1); no leakage columns; no nulls
<code>validate_features</code>	After FE	All 4 engineered features present
<code>validate_feature_names</code>	After preprocessor fit	Feature name extraction succeeds
<code>validate_prediction_output</code>	After predict	Schema correct; probabilities in $[0, 1]$; valid risk tiers

Validation raises `ValidationError` on failure, halting the pipeline before incorrect data propagates downstream.

3.2 Output Validation

Prediction outputs are validated to ensure:

- All required columns (`appointment_id`, `no_show_probability`, `risk_tier`) are present.
- Probabilities lie in $[0, 1]$.
- Risk tiers are one of `HIGH_RISK`, `MODERATE_RISK`, or `LOW_RISK`.

—

4 Integrated Pipeline (HealthConnectPipeline)

The `HealthConnectPipeline` class is the central Week 6 deliverable. It wraps the full workflow:

1. **Load:** Read CSV, validate raw schema.
2. **Clean:** Binary filter, target encoding, leakage column drop, imputation.
3. **Feature engineer:** 4 derived behavioural features.
4. **Split:** Stratified 80/20 train/test.
5. **Preprocess:** Fit `ColumnTransformer` on training data, transform both splits.
6. **Train:** Cross-validate and fit the best model.
7. **Evaluate:** Compute metrics on held-out test set.
8. **Predict:** Generate risk-stratified predictions with recommended actions.

4.1 Usage

```
pipe = HealthConnectPipeline(threshold=0.50)
results = pipe.run()                # full pipeline
pred = pipe.predict_single(record)  # single prediction
preds = pipe.predict_batch(df)      # batch predictions
pipe.save_artifacts("artifacts")    # persist to disk
```

4.2 Prediction Output Schema

Table 2: Prediction Output Schema

Field	Type	Description
<code>appointment_id</code>	String	Appointment identifier
<code>no_show_probability</code>	Float [0, 1]	$P(\text{No-Show})$
<code>risk_tier</code>	String	HIGH / MODERATE / LOW
<code>recommended_action</code>	String	Automated engagement workflow

Risk tier thresholds (from Week 4 architecture):

- **High Risk** ($P \geq 0.70$): WhatsApp confirmation + buffer slot.
- **Moderate Risk** ($0.40 \leq P < 0.70$): SMS reminder requesting confirmation.
- **Low Risk** ($P < 0.40$): Standard email notification.

—

5 Improved Model Configurations

Based on Week 5 error analysis and cross-track input from Data Science, the Week 6 improved models use tuned hyperparameters:

All models retain `class_weight='balanced'` for cost-sensitive recall focus.

—

Table 3: Week 5 vs Week 6 Model Hyperparameters

Model	Week 5	Week 6
Random Forest	200 trees, depth 10	500 trees, depth 12, leaf=5
Gradient Boosting	200 trees, lr 0.1	300 trees, lr 0.05, subsample 0.8
LightGBM	200 trees, lr 0.1	400 trees, lr 0.05, colsample 0.8
Logistic Regression	$C = 1.0$	$C = 0.5$, iter 2000

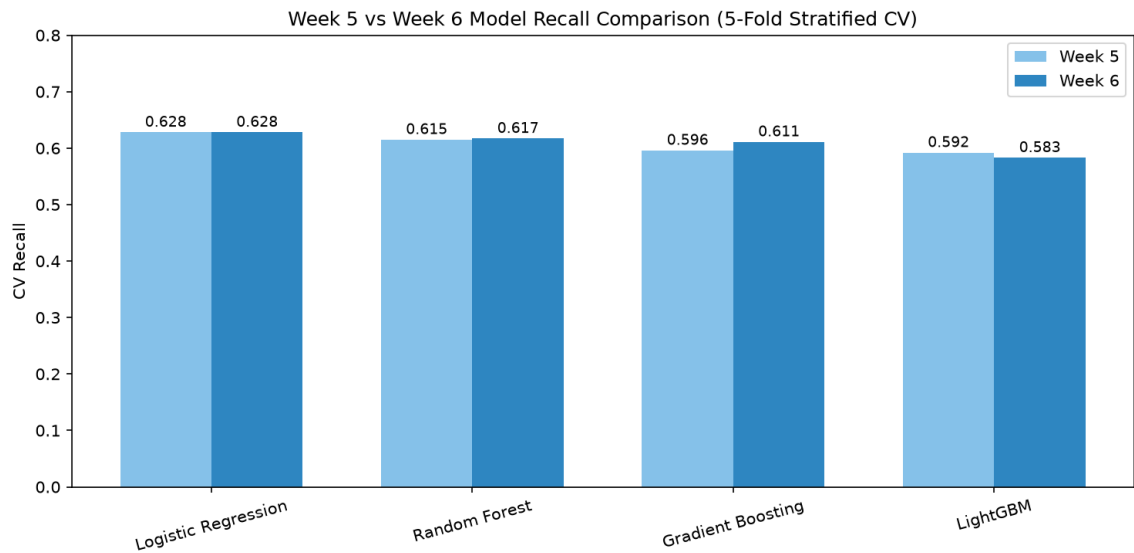


Figure 1: Week 5 vs Week 6 CV Recall comparison across four common models.

Table 4: Baseline Model Comparison: Week 5 vs Week 6 (5-Fold Stratified CV Recall)

Model	Week 5 Recall	Week 6 Recall
Logistic Regression	0.628	0.628
Random Forest	0.615	0.617
Gradient Boosting	0.596	0.611
LightGBM	0.592	0.583

6 Results & Model Comparison

Gradient Boosting improved by +0.015 in recall under the tuned configuration. LightGBM recalled slightly (−0.009), likely due to the stronger regularisation from `min_child_samples=20` and requires further tuning.

6.1 Best Model (Held-Out Test Set)

—

7 Cross-Track Collaboration

Track collaborated with: Data Science

Project dependency: Data Science produces the candidate model and error analysis that informs which model and hyperparameters the ML Engineering pipeline integrates.

Table 5: Best Model (Logistic Regression) — Held-Out Test Set ($N = 948$)

Metric	Value
Accuracy	0.617
Precision	0.621
Recall	0.647
F1	0.634
F2	0.642
ROC-AUC	0.664

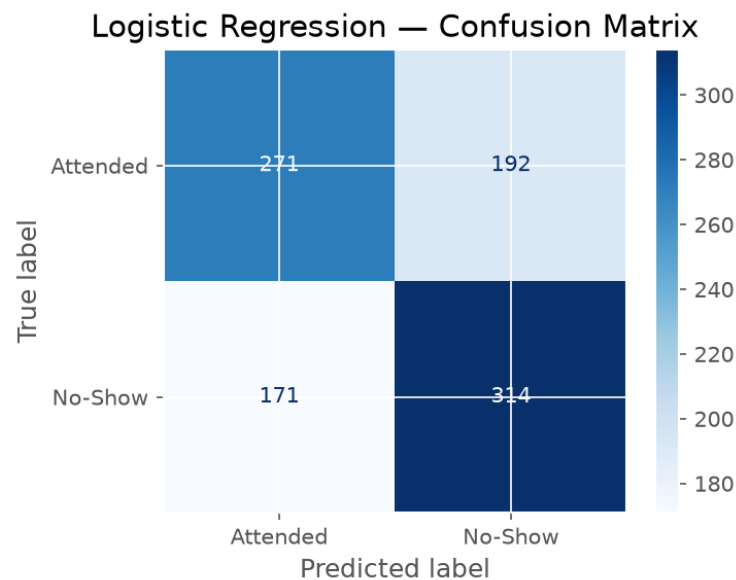


Figure 2: Confusion matrix for the best model on the held-out test set.

Information/output received: Candidate model (Random Forest), error analysis identifying false negatives as the primary concern, feature importance confirming `booking_lead_days` as the dominant signal.

Information/output provided: Technical constraints on model interface compatibility, pre-processing format requirements, and pipeline input/output schema specifications.

Integration activity completed: Integrated the Data Science candidate model configuration into `get_improved_models()` and validated the full pipeline flow.

What changed as a result: Week 6 improved models use tuned hyperparameters informed by DS error analysis; the pipeline validates prediction output against the agreed schema; logging provides execution traceability.

Evidence: This notebook, `pipeline/model_integration.py`, `pipeline/validation.py`, and the integration test suite (`tests/test_integration.py`).

8 Testing Evidence

Integration tests verify:

- Pipeline fits and produces a trained model.

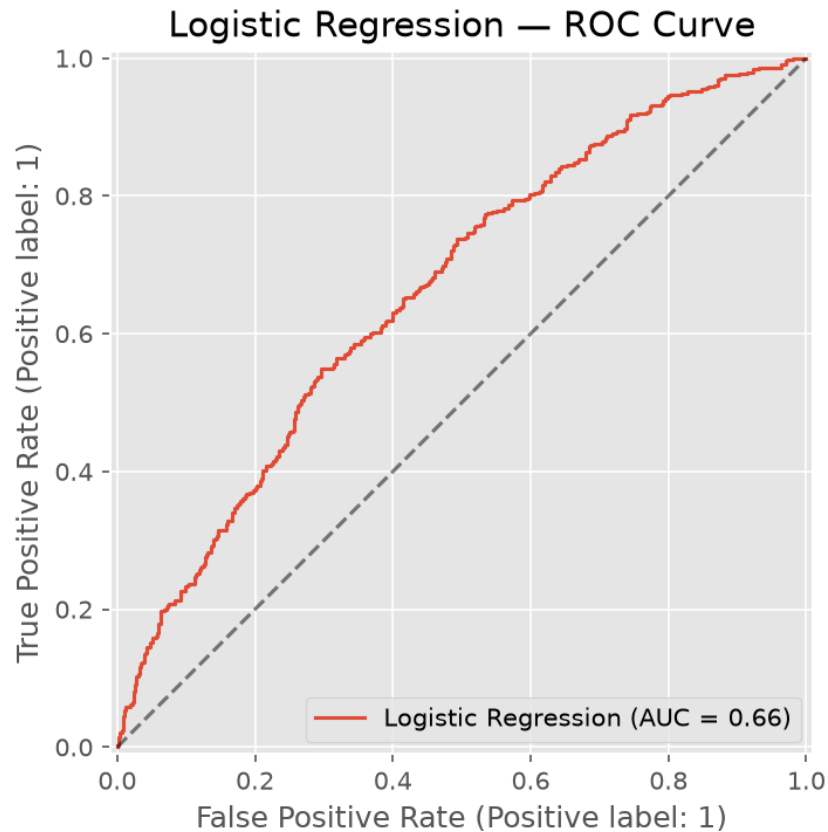


Figure 3: ROC curve for the best model (AUC = 0.664).

Table 6: Test Suite Summary

Test File	Tests	Status
test_pipeline.py (Week 5)	6 unit tests	All passing
test_pipeline.py (Week 6 additions)	6 validation tests	All passing
test_integration.py	5 integration tests	All passing
Total	17	All passing

- Feature names are extracted correctly.
- Single-record prediction returns valid output.
- Prediction output passes schema validation.
- Pipeline raises `ValidationError` if called before fitting.

9 Assumptions, Limitations, Risks & Dependencies

9.1 Resolved from Week 5

- Input/output validation checks now enforce schema correctness.
- Structured logging replaces print statements for execution traceability.

- Integrated pipeline class (`HealthConnectPipeline`) provides a single entry point.
- Integration tests verify preprocessing-model interaction.

9.2 Remaining

- `GroupKFold` by `patient_id` is still deferred to Week 7 for rigorous leakage control.
- Production deployment (FastAPI, Docker) is deferred to Week 8.
- Drift monitoring is not yet implemented.
- LightGBM recall dropped under tuned hyperparameters; requires further tuning.

9.3 New Issues Discovered

- The pipeline holds the fitted model in memory; `save_artifacts()` persists to disk but end-to-end loading and prediction from saved artefacts requires Week 7 validation.
 - Stratified split still does not fully control repeat-patient leakage.
-

10 Recommended Focus for Week 7

1. End-to-end validation with `GroupKFold` grouped by `patient_id`.
2. Threshold tuning for F_2 -score optimisation.
3. Model artefact loading and prediction on unseen data from saved files.
4. FastAPI inference service integration (containerised).
5. MLflow experiment tracking setup.
6. Evidently AI drift monitoring baseline.